

STUDIO DELLE TECNICHE DI
CLASSIFICAZIONE DI GENERI MUSICALI
Progetto Machine Learning

Mattero Romeo e Davide Orienti

535005 - 533107

Indice Generale

1	Introduzione	4
2	Capitolo 1: Studio del Dataset	4
3	Studio delle Features	5
3.1	Selezione delle features	6
3.1.1	Correlazione tra features	6
3.2	Data Argumentation	7
3.3	Standardizzazione delle features	8
4	Tecniche di machine learning	8
5	Logistic regression	8
5.1	Test effettuati con features di 3 secondi	9
5.2	Ottimizzazione degli iperparametri	10
5.3	Possibili spiegazioni delle differenze osservate	10
5.4	Modello Base	10
5.5	Standardizzazione	11
5.6	Grid Search per l'Ottimizzazione	11
5.7	Conclusione	12
6	Random forest	12
6.1	Modello Base senza Standardizzazione	12
6.2	Applicazione della Standardizzazione	13
6.3	Ottimizzazione con Grid Search	13
6.4	Risultati e Conclusioni	14
6.4.1	Dataset Complesso e Non Lineare	14
7	K-Nearest Neighbors	15
7.1	Modello Base senza Standardizzazione	15
7.1.1	Cause della Ridotta Accuratezza	16
7.1.2	Implicazioni sulla Matrice di Confusione	16
7.1.3	Conclusione	17
7.2	Applicazione della Standardizzazione	17
7.3	Ottimizzazione con Grid Search	19
7.4	Risultati e Conclusioni	19
8	Support Vector Machine	20
8.1	Fase 1: Addestramento Senza Standardizzazione	20
8.1.1	Vantaggi e Limiti	20
8.2	Fase 2: Addestramento con Standardizzazione	21
8.2.1	Risultati e Benefici	21
8.3	Fase 3: Ottimizzazione con Grid Search	22
8.3.1	Vantaggi della Grid Search	22
8.4	Conclusioni	23

9	Extreme Gradient Boosting	24
9.1	Fase 1: Addestramento Senza Standardizzazione	24
9.1.1	Vantaggi e Limiti	24
9.2	Fase 2: Addestramento con Standardizzazione	24
9.2.1	Risultati	25
9.3	Fase 3: Ottimizzazione con Grid Search	25
9.3.1	Vantaggi della Grid Search	25
9.4	Conclusioni	26
10	Semplice Rete Neurale Implementata	26
10.1	Pre-elaborazione dei Dati	26
10.2	Implementazione dei Modelli di Rete Neurale	27
10.2.1	Modello 1 - Architettura di Base	27
10.2.2	Modello 2 - Aggiunta di Dropout	28
10.2.3	Modello 3 - Batch Normalization e Ottimizzazione Avanzata . . .	28
10.3	Valutazione e Confronto dei Risultati	29
10.4	Conclusioni	29

1 Introduzione

Questo progetto si basa sullo studio dell'abilità di un sistema di Machine Learning di identificare e classificare il genere musicale di una traccia audio. Il dataset utilizzato per questo task è il noto dataset GTZAN (<https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification>), composto da dieci generi musicali distinti, ciascuno composto da 100 tracce audio. I generi musicali che si trovano all'interno di questo dataset sono: blues, classical, country, hiphop, disco, jazz, metal, rock, pop e reggae. Ogni traccia audio è della durata di circa 30 secondi e rappresenta una composizione tipica del genere a cui appartiene, rendendo il dataset ideale per le applicazioni di classificazione. Nel presente studio, verrà analizzata e confrontata l'efficacia di diverse tecniche di estrazione delle feature e di modelli di classificazione, confrontando i risultati ottenuti in termini di accuratezza e robustezza. Come prima analisi si è andato a studiare la distribuzione e caratteristiche dei dati all'interno del dataset. Dopodichè sono state provate e valutate alcune tecniche di feature engineering. Dopo aver effettuato queste analisi sono state provate alcune delle tecniche studiate durante il corso come Random Forest, Logistic Regression, SVM, KNN e sono stati confrontati i risultati ottenuti. Infine è stata sviluppata una rete neurale per tentare di migliorare i dati ottenuti dalle tecniche di classificazione eseguite in precedenza.

2 Capitolo 1: Studio del Dataset

Il dataset scelto per questo progetto, noto come GTZAN (<https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification>), rappresenta una delle risorse più rilevanti e consolidate nell'ambito della classificazione musicale, fornendo un'ampia gamma di dati utili per l'analisi dei generi musicali. Questo dataset è strutturato in dieci generi musicali: blues, classical, country, hiphop, disco, jazz, metal, rock, pop e reggae. Ogni genere contiene esattamente 100 file audio della durata di 30 secondi ciascuno, garantendo una struttura bilanciata e uniforme che facilita il processo di analisi e interpretazione. L'uniformità del dataset riduce il rischio di sbilanciamenti tra le classi, un aspetto che potrebbe influire negativamente sui risultati di classificazione. Oltre ai file audio, GTZAN contiene ulteriori cartelle con gli spettrogrammi estratti e un insieme di features derivate dai file audio originali. Queste componenti aggiuntive permettono di approcciare il dataset da diverse prospettive analitiche. Per la prima fase del progetto, l'attenzione si è focalizzata sulle features estratte, utilizzando queste come base per costruire modelli di classificazione. Un'ulteriore analisi della composizione del dataset ha evidenziato una distribuzione uniforme tra i generi: ciascun genere. Oltre ai dati di 30 secondi, GTZAN include anche una rappresentazione dei campioni in formato ridotto. In questo file, ogni traccia è stata suddivisa in dieci segmenti di 3 secondi ciascuno, portando il numero totale dei campioni a 10.000 e incrementando la granularità del dataset. Tale suddivisione permette di aumentare il numero complessivo dei dati forniti al modello, aspetto che può influire positivamente sulle prestazioni complessive della classificazione. Poiché il dataset è etichettato in modo completo, si è scelto un approccio di apprendimento supervisionato. Questo approccio permette di sfruttare al massimo le informazioni presenti nelle etichette, supportando il modello nella distinzione tra i generi musicali.

Table 1: Caratteristiche del dataset GTZAN

Caratteristica	Descrizione
Nome del Dataset	GTZAN (Music Genre Classification Dataset).
Generi Musicali	10 generi: blues, classical, country, hiphop, disco, jazz, metal, rock, pop, reggae.
Numero di Campioni	100 file per genere, ciascuno della durata di 30 secondi.
Bilanciamento	Dataset bilanciato: 1.000 campioni totali, 100 per ciascun genere.
Features Estratte	Contiene features audio e spettrogrammi derivati dai file originali.
Operazione di Segmentazione	Ogni traccia è suddivisa in 10 segmenti da 3 secondi, per un totale di 10.000 campioni.
Rappresentazioni Dati	File audio originali, spettrogrammi, e features estratte.
Tecnica di Apprendimento	Approccio supervisionato utilizzando le etichette di genere.
Scopo	Analisi e classificazione dei generi musicali.

3 Studio delle Features

Il dataset GTZAN include un insieme di 60 **features** estratte dai file audio, ciascuna delle quali fornisce una rappresentazione specifica delle caratteristiche acustiche dei brani. Queste *features* sono state scelte per facilitare il modello di apprendimento nella distinzione tra i vari generi musicali. Di seguito una panoramica delle *features* contenute nel dataset:

- **filename**: Nome del file audio, utilizzato principalmente per identificare il campione e non rilevante per la classificazione.
- **length**: Durata del file audio in secondi.
- **chroma_stft_mean** e **chroma_stft_var**: Media e varianza della trasformata di Fourier con componenti cromatiche, utili per l'analisi delle tonalità e degli accordi.
- **rms_mean** e **rms_var**: Media e varianza del Root Mean Square (RMS) dell'ampiezza, indicano l'intensità sonora media.
- **spectral_centroid_mean** e **spectral_centroid_var**: Media e varianza del centroide spettrale; un valore elevato indica una predominanza di frequenze alte.
- **spectral_bandwidth_mean** e **spectral_bandwidth_var**: Media e varianza della larghezza di banda spettrale, che indica l'ampiezza della gamma di frequenze attorno al centroide spettrale.
- **rolloff_mean** e **rolloff_var**: Media e varianza della frequenza di roll-off, ossia il punto in cui si accumula l'85% dell'energia spettrale.
- **zero_crossing_rate_mean** e **zero_crossing_rate_var**: Media e varianza della frequenza di attraversamento dello zero, utili per distinguere tra suoni rumorosi e tonalità più morbide.

- **harmony_mean** e **harmony_var**: Media e varianza dell'armonia, descrivono la componente armonica del suono e sono cruciali per l'identificazione delle tonalità.
- **perceptr_mean** e **perceptr_var**: Media e varianza del contrasto percepibile, utilizzato per distinguere le strutture armoniche dal rumore.
- **tempo**: Velocità del brano espressa in battiti per minuto (BPM), utile nella classificazione di generi ritmici.
- **mfcc1_mean - mfcc20_mean** e **mfcc1_var - mfcc20_var**: Media e varianza dei primi 20 coefficienti Mel-Frequency Cepstral Coefficients (MFCC), che rappresentano la forma spettrale del segnale audio su una scala non lineare che riflette la percezione umana delle frequenze. I MFCC sintetizzano informazioni cruciali sull'intonazione e sul timbro del brano e sono fondamentali per la classificazione.
- **label**: Etichetta che identifica il genere musicale del file audio, utilizzata come variabile target per il modello.

Queste *features* forniscono una rappresentazione completa dei file audio, comprendono sia aspetti spettrali (come le frequenze e l'intensità) sia aspetti temporali (come il ritmo e l'armonia). Grazie a queste caratteristiche, è possibile distinguere i generi musicali poiché ciascun genere tende a presentare schemi distinti in termini di frequenze dominanti, intensità, ritmo e armoniche.

La presenza dei campioni suddivisi in segmenti da 3 secondi, per un totale di 10.000 file, migliora ulteriormente la granularità del dataset consentendo al modello di analizzare le variazioni temporali interne di ciascun file, supportando un'analisi più dettagliata del contenuto sonoro.

Infine, dopo aver analizzato e compreso la struttura delle *features*, abbiamo intrapreso l'applicazione di tecniche di *feature engineering* per ottimizzare ulteriormente le performance del modello e cercare migliorare i risultati di classificazione.

3.1 Selezione delle features

3.1.1 Correlazione tra features

Come prima operazione, ci siamo concentrati sull'analisi delle correlazioni tra le varie features del dataset per identificare eventuali coppie di features con correlazioni elevate. Siamo quindi andati a creare la matrice di correlazione. Studiando i valori della matrice sappiamo che valori prossimi a 1 o a -1 indicano una forte correlazione tra due features, segnalando una relazione lineare diretta o inversa. Quando due features risultano esattamente correlate (valore di correlazione pari a 1 o -1), è possibile eliminare una delle due senza perdita di informazione significativa, semplificando così il modello e riducendo la dimensionalità.

3.3 Standardizzazione delle features

Come ultima tecnica nell'ambito del feature engineering che abbiamo utilizzato è stata la standardizzazione. Questo processo consiste nel trasformare i dati in modo che abbiano una media pari a zero e una deviazione standard pari a uno. Tale operazione non solo rende le features comparabili tra loro, ma accelera anche la convergenza degli algoritmi di apprendimento, facilitando l'addestramento dei modelli. Nel nostro studio, sono state prese in considerazione quattro diverse tecniche di standardizzazione, ognuna delle quali ha caratteristiche specifiche e può essere più o meno adatta in base al contesto, alla natura dei dati ed al modello utilizzato. Nel nostro caso specifico tra le possibili standardizzazioni Robust Scaler, Min-Max Scaler, Power Transformer abbiamo scelto di usare Standard Scaler. Questo approccio è efficace quando i dati seguono una distribuzione normale. Abbiamo notato immediatamente dei miglioramenti.

4 Tecniche di machine learning

Nella fase attuale del progetto, ci siamo focalizzati sull'analisi delle varie tecniche di *machine learning* esaminate durante il corso, confrontando le loro performance. In particolare, abbiamo effettuato un confronto tra gli algoritmi utilizzati con il dataset composto da campioni da 3 secondi.

Per ogni tecnica di *machine learning* adottata, abbiamo estrapolato diverse metriche di valutazione, tra cui:

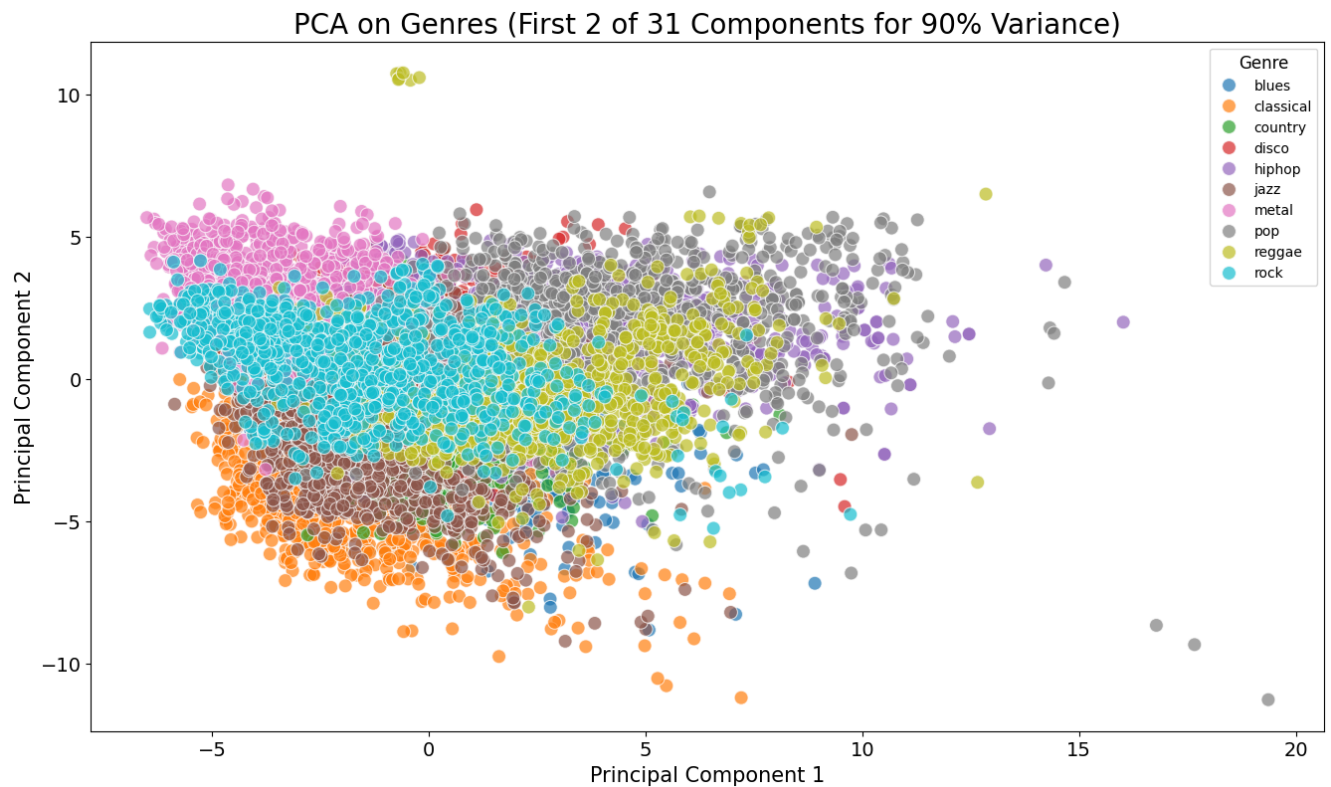
- **Precision**
- **Recall**
- **Accuracy**
- **F1 / F2**

Le tecniche di classificazione utilizzate Sono state:

- **Logistic Regression**
- **Random Forest**
- **Knn**
- **SVM**
- **XGBoost**
- **Neural Network**

5 Logistic regression

Come primo approccio alla classificazione musicale, è stata scelta la logistic regression considerando la sua semplicità di utilizzo e la bassa complessità computazionale già sappiamo che non avremo un buon risultato. Questo perché è adatto per catturare pattern



lineari e nel nostro caso specifico cio è molto difficile non avendo una classificazione binaria ma avendo ben 10 classi diversi. Infatti andando a stampare le prime 2 componenti principali tramite PCA vediamo con il grafico il seguente risultato.

Quindi possiamo affermare che non riusciamo a fare una corretta separazione lineare dei generi musicali e questo lo vediamo bene con i risultati ottenuti dall'esecuzione dell'algoritmo.

5.1 Test effettuati con features di 3 secondi

Come primo passo abbiamo caricato il dataset con i file csv che rappresentano i campioni di 3 secondi.

```

-----STATISTICHE TEST-----
Accuratezza: 0.5040
Precision: 0.4943
Recall: 0.5069
F1-score: 0.4832
F2-score: 0.4930
-----STATISTICHE TRAINING-----
Accuratezza: 0.5058
Precision: 0.4911
Recall: 0.5047
F1-score: 0.4855
F2-score: 0.4940

```

Dopodiché abbiamo proceduto con l'esecuzione dell'algoritmo e analizzato i risultati applicando una dovuta standardizzazione.

```
-----STATISTICHE TEST-----
Accuratezza: 0.7212
Precision: 0.7170
Recall: 0.7218
F1-score: 0.7161
F2-score: 0.7188
-----STATISTICHE TRAINING-----
Accuratezza: 0.7270
Precision: 0.7225
Recall: 0.7267
F1-score: 0.7233
F2-score: 0.7251
```

Nel complesso, i risultati mostrano che il modello ha raggiunto prestazioni stabili e coerenti, con valori di metriche che non si discostano significativamente l'uno dall'altro andando ad escludere comportamenti come underfitting o overfitting. Questo suggerisce un modello ben bilanciato e relativamente robusto.

5.2 Ottimizzazione degli iperparametri

Dopo aver studiato i risultati, per migliorarli, si è pensato di procedere andando ad ottimizzare gli iperparametri. La tecnica di ricerca utilizzata è stata la Grid Search. E' stata scelta la grid search perché esamina tutte le combinazioni possibili dei valori specificati per ciascun iperparametro. Assicura di individuare l'insieme di iperparametri ottimale all'interno dello spazio di ricerca, ma può essere lenta su dataset di grandi dimensioni o con molti iperparametri. Dopo aver definito il set di iperparametri da esplorare e individuato quelli ottimali, abbiamo eseguito l'algoritmo.

```
-----STATISTICHE TEST-----
Accuratezza: 0.7362
Precision: 0.7317
Recall: 0.7371
F1-score: 0.7330
F2-score: 0.7351
-----STATISTICHE TRAINING-----
Accuratezza: 0.7416
Precision: 0.7392
Recall: 0.7414
F1-score: 0.7399
F2-score: 0.7407
```

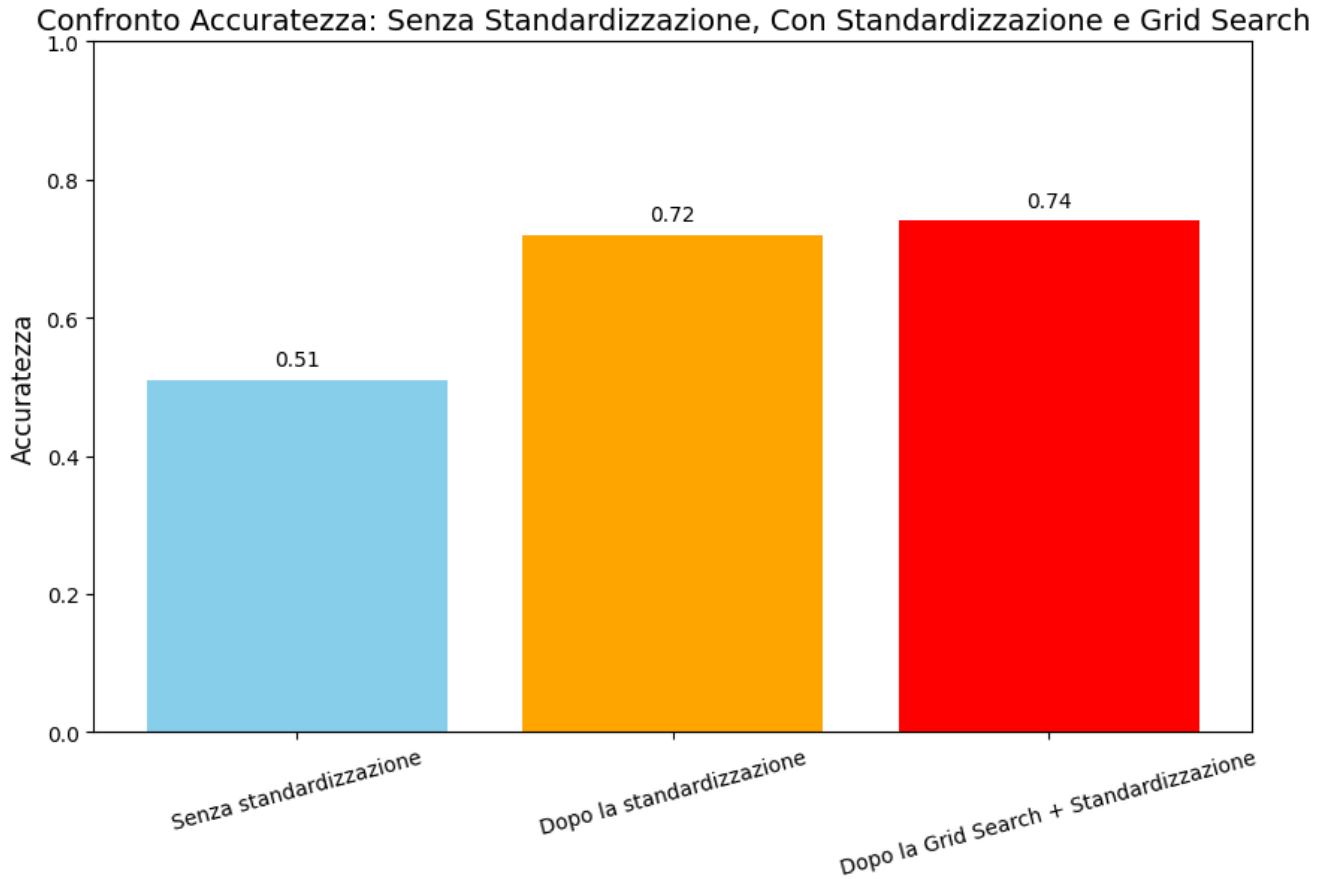
5.3 Possibili spiegazioni delle differenze osservate

Il modello di Logistic Regression è stato progressivamente ottimizzato attraverso tre fasi:

5.4 Modello Base

Il modello iniziale, senza alcun preprocessing, ha presentato alcune problematiche:

- Differenze di scala tra le feature, causando difficoltà nella convergenza.



- Feature dominanti che influenzano eccessivamente il modello.
- Scarsa generalizzazione, con possibili bias nei pesi.

5.5 Standardizzazione

L'applicazione della standardizzazione (*StandardScaler*) ha migliorato le prestazioni grazie a:

- Convergenza più rapida dell'ottimizzatore.
- Pesi più bilanciati tra le feature.
- Migliore stabilità e generalizzazione del modello.

5.6 Grid Search per l'Ottimizzazione

L'uso della Grid Search ha permesso di trovare i migliori iperparametri, ottenendo:

- Selezione ottimale del parametro di regolarizzazione C , evitando overfitting e underfitting.
- Maggiore bilanciamento tra bias e varianza.
- Migliore accuratezza e stabilità nei risultati.

5.7 Conclusione

L'ottimizzazione progressiva ha portato a un modello più robusto ed efficace, con migliori capacità di generalizzazione e predizione. Per ottenere ulteriori miglioramenti, si suggerisce di esplorare:

- Modelli più complessi: Come le support vector machine (SVM) o le random forest, in grado di gestire relazioni non lineari.

6 Random forest

Random Forest è un algoritmo di apprendimento supervisionato che combina molti alberi decisionali per fare previsioni. Questo metodo è particolarmente efficace per la classificazione perché tende a essere robusto agli outlier. Quindi per il nostro task potrebbe essere una buona soluzione alternativa.

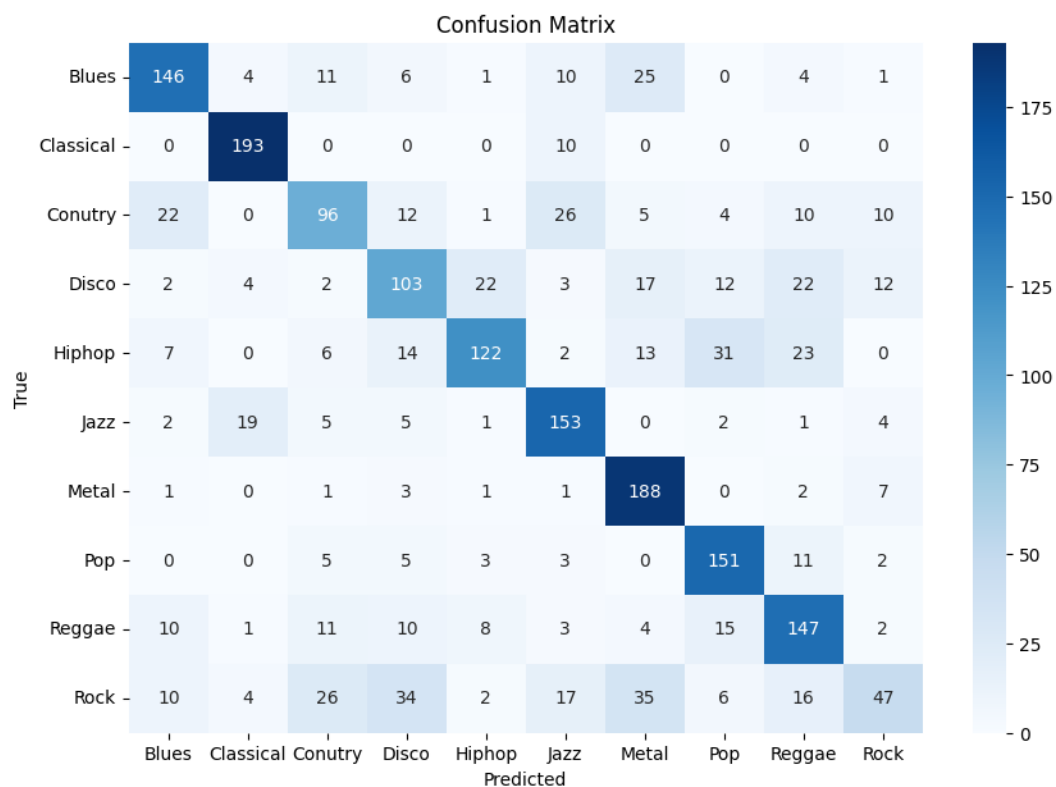
In questa sezione, presentiamo il processo di sviluppo, valutazione e ottimizzazione del modello *Random Forest* per la classificazione dei generi musicali, evidenziando l'impatto delle diverse tecniche applicate.

6.1 Modello Base senza Standardizzazione

Abbiamo inizialmente addestrato un modello *Random Forest* con *100 alberi* e una *profondità massima di 5*, senza applicare alcuna pre-elaborazione sui dati. Questo approccio presenta alcune criticità:

- Questo modello è robusto alla diversità delle scale dei vari valori delle features.
- La convergenza del modello potrebbe essere meno efficace.
- Potenziali problemi di squilibrio tra feature con valori numerici molto diversi.

Dalla valutazione iniziale, abbiamo ottenuto un'accuratezza di test inferiore rispetto a modelli più ottimizzati.



```

-----STATISTICHE TEST-----
Accuratezza: 0.6151
Precision: 0.6069
Recall: 0.6146
F1-score: 0.5934
F2-score: 0.6029
-----STATISTICHE TRAINING-----
Accuratezza: 0.6436
Precision: 0.6404
Recall: 0.6435
F1-score: 0.6280
F2-score: 0.6347

```

6.2 Applicazione della Standardizzazione

Abbiamo quindi applicato la standardizzazione ai dati di input, normalizzandone la distribuzione tramite *StandardScaler*.

Come ci si aspettava, le prestazioni sono rimaste molto simili alle prestazioni precedenti.

6.3 Ottimizzazione con Grid Search

Per massimizzare le prestazioni del modello, abbiamo eseguito una ricerca degli iperparametri utilizzando *Grid Search* con validazione incrociata. I parametri ottimizzati includono:

- Numero di alberi nella foresta ($n_estimators$).
- Profondità massima degli alberi (max_depth).

```

-----STATISTICHE TEST-----
Accuratezza: 0.6201
Precision: 0.6081
Recall: 0.6201
F1-score: 0.5979
F2-score: 0.6080
-----STATISTICHE TRAINING-----
Accuratezza: 0.6433
Precision: 0.6411
Recall: 0.6431
F1-score: 0.6267
F2-score: 0.6335

```

- Numero minimo di campioni richiesti per dividere un nodo (*min_samples_split*).
- Numero minimo di campioni richiesti in una foglia (*min_samples_leaf*).

Dopo l'ottimizzazione, abbiamo selezionato il miglior modello tra quelli con una differenza di accuratezza tra training e test inferiore al **10%**, evitando fenomeni di overfitting.

```

Fitting 5 folds for each of 81 candidates, totalling 405 fits
Training Accuracy: 0.7187187187187187
Testing Accuracy: 0.6791791791791791
Best Parameters: {'max_depth': 6, 'min_samples_leaf': 4, 'min_samples_split': 2, 'n_estimators': 200}
La differenza tra l'accuracy di training e di test è entro il 10%.

```

6.4 Risultati e Conclusioni

L'accuratezza del modello ottimizzato ha raggiunto un valore pari a **0.72**, con una generalizzazione equilibrata tra training e test. La Standardizzazione non ha portato dei miglioramenti perché:

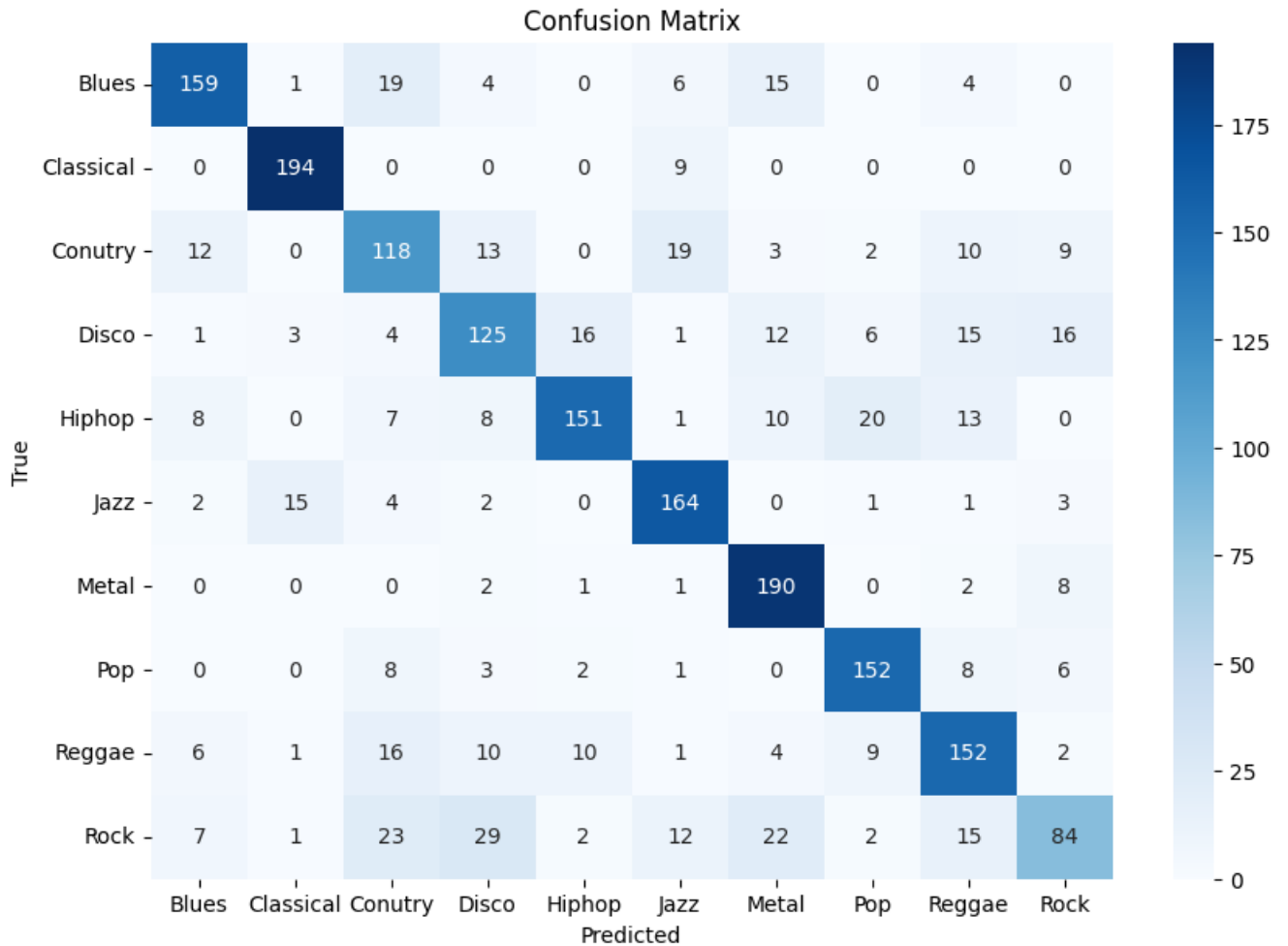
- **Non è sensibile alla scala delle feature**, in quanto prende decisioni basate su soglie di split e non su misure di distanza.
- **Utilizza partizioni binarie** per suddividere i dati, quindi trasformazioni come la standardizzazione non modificano il modo in cui vengono generati gli split.

Di conseguenza, l'operazione di standardizzazione non ha portato miglioramenti nel modello come ci si aspettava.

6.4.1 Dataset Complesso e Non Lineare

Il dataset potrebbe contenere relazioni complesse e altamente non lineari tra le feature e le classi di output. Sebbene la Random Forest sia in grado di catturare relazioni non lineari, non ha la capacità di apprendere rappresentazioni gerarchiche astratte come le reti neurali. Pertanto, le reti neurali potrebbero essere più adatte a questo problema.

L'uso della selezione dei parametri e ottimizzazione degli iperparametri ha portato a un modello più stabile e affidabile rispetto alla versione iniziale. Inoltre, la matrice di confusione ha confermato una distribuzione più bilanciata degli errori tra le classi.



L'approccio adottato dimostra l'importanza della pre-elaborazione dei dati e dell'ottimizzazione degli iperparametri nella costruzione di modelli di *Machine Learning* robusti e generalizzabili.

7 K-Nearest Neighbors

In questa sezione analizziamo il processo di sviluppo e ottimizzazione del modello *K-Nearest Neighbors* (KNN) per la classificazione dei generi musicali. Il modello è stato valutato in tre fasi principali:

1. Modello base senza standardizzazione.
2. Applicazione della standardizzazione.
3. Ottimizzazione con ricerca degli iperparametri (*Grid Search*).

7.1 Modello Base senza Standardizzazione

Nella fase iniziale, il modello *K-Nearest Neighbors* (KNN) è stato addestrato senza applicare alcuna standardizzazione alle feature. L'accuratezza ottenuta è risultata inferiore rispetto alle fasi successive, evidenziando alcune problematiche strutturali che influenzano negativamente le prestazioni del modello.

7.1.1 Cause della Ridotta Accuratezza

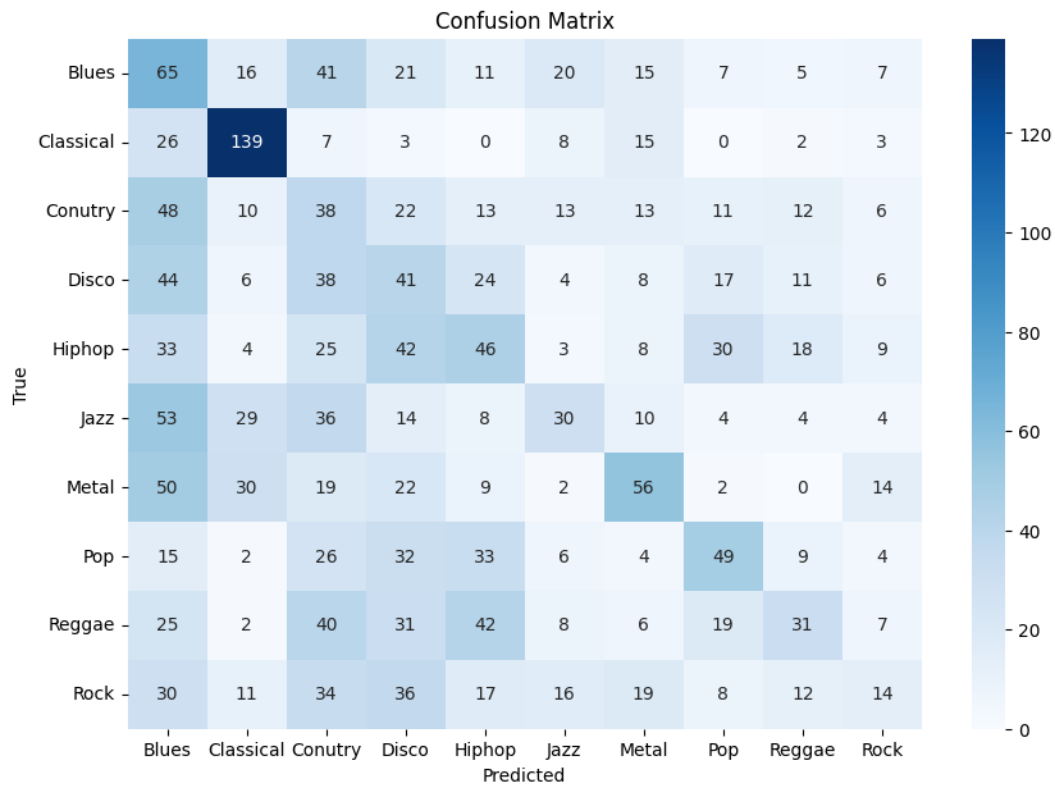
La bassa performance del modello prima della standardizzazione può essere attribuita a diversi fattori:

- **Scala delle Feature Non Uniforme:** KNN si basa sulle distanze tra punti nel dataset. Feature con ordini di grandezza diversi dominano il calcolo della distanza, causando bias nella classificazione.
- **Influenza delle Feature con Maggiore Scala:** Le feature con valori numerici più alti hanno un impatto maggiore sul modello, riducendo la rilevanza delle altre feature.
- **Minore Stabilità del Modello:** Senza standardizzazione, il comportamento del modello è più sensibile alla scelta di k e della metrica di distanza, rendendo le predizioni meno affidabili.

```
-----STATISTICHE TEST-----  
Accuratezza: 0.2548  
Precision: 0.2689  
Recall: 0.2540  
F1-score: 0.2495  
F2-score: 0.2497  
-----STATISTICHE TRAINING-----  
Accuratezza: 0.5533  
Precision: 0.5950  
Recall: 0.5536  
F1-score: 0.5449  
F2-score: 0.5440
```

7.1.2 Implicazioni sulla Matrice di Confusione

L'analisi della matrice di confusione mostra un'elevata confusione tra alcune classi, in particolare tra generi musicali con caratteristiche simili. Ciò suggerisce che le distanze calcolate dal modello non riflettono adeguatamente la separazione tra le classi a causa delle discrepanze nelle scale delle feature.



7.1.3 Conclusione

Questi risultati evidenziano l'importanza della pre-elaborazione dei dati prima dell'addestramento. La standardizzazione si rivela un passaggio fondamentale per rendere il modello più equo e bilanciato, migliorando la qualità della classificazione e la capacità di generalizzazione. L'implementazione della standardizzazione nelle fasi successive ha dimostrato un aumento significativo dell'accuratezza e della stabilità del modello.

L'accuratezza iniziale è stata calcolata e confrontata con le versioni successive.

7.2 Applicazione della Standardizzazione

Successivamente, abbiamo applicato la standardizzazione ai dati utilizzando *StandardScaler* per uniformare la distribuzione delle feature. Questo passaggio ha portato ai seguenti benefici:

- Equilibrio tra le feature, evitando che alcune abbiano maggiore influenza sulle distanze.
- Miglioramento della stabilità del modello e della sua capacità di generalizzazione.
- Aumento dell'accuratezza sul test set rispetto al modello non standardizzato.

Il miglioramento delle prestazioni è stato analizzato mediante un confronto grafico tra accuratezza prima e dopo la standardizzazione.

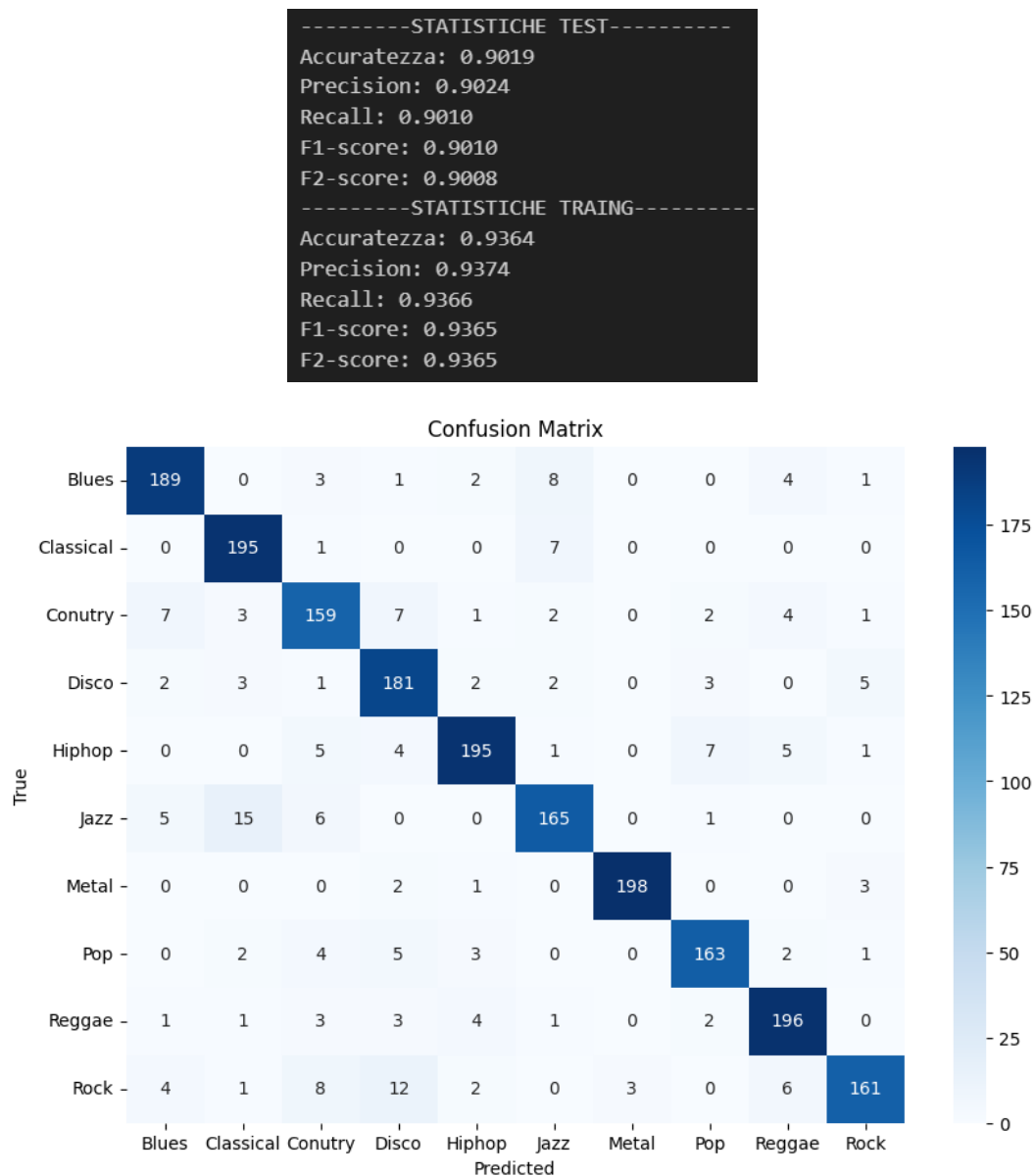


Figure 2: Matrice di confusione molto piu omogenea e migliore

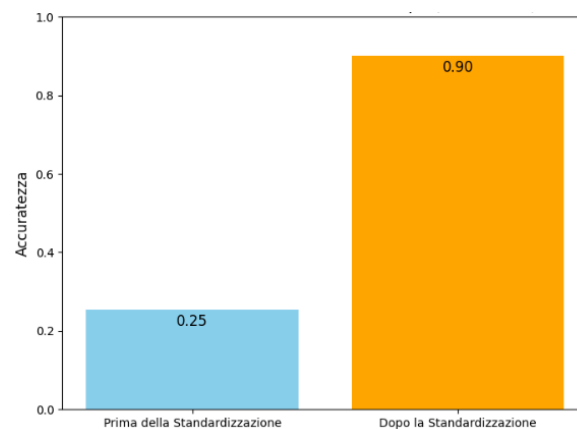


Figure 3: Confronto dei risultati prima e dopo la standardizzazione

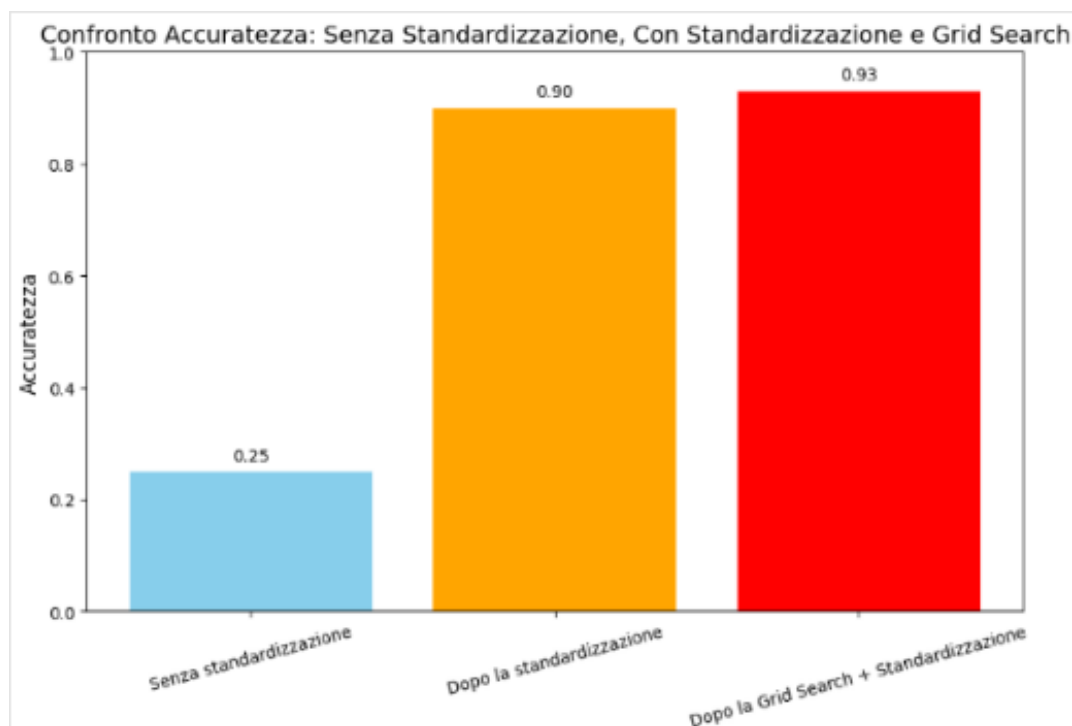
7.3 Ottimizzazione con Grid Search

Per migliorare ulteriormente il modello, abbiamo eseguito una ricerca degli iperparametri utilizzando *Grid Search* con validazione incrociata. I parametri ottimizzati includono:

- Numero di vicini (k), valutato tra $\{3, 5, 7\}$.
- Metodo di pesatura dei vicini (*uniform* vs. *distance*).
- Distanza utilizzata (*euclidean* vs. *manhattan*).

Dopo l'ottimizzazione, abbiamo selezionato il miglior modello tra quelli con una differenza di accuratezza tra training e test inferiore al **10%**, per evitare overfitting.

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits
Migliori parametri trovati: {'metric': 'manhattan', 'n_neighbors': 3, 'weights': 'distance'}
Training Accuracy: 0.9707207207207207
Testing Accuracy: 0.9269269269269269
Best Parameters: {'metric': 'manhattan', 'n_neighbors': 3, 'weights': 'uniform'}
```



7.4 Risultati e Conclusioni

Il miglior modello ottenuto utilizza la distanza *manhattan*, $k = 3$ e pesatura basata sulla distanza. L'accuratezza finale è risultata superiore rispetto ai modelli precedenti, con un bilanciamento ottimale tra training e test. L'uso della standardizzazione e della selezione dei parametri ha portato a un modello più stabile e performante. Inoltre, la matrice di confusione ha confermato una distribuzione più bilanciata degli errori tra le classi.

Questo approccio evidenzia l'importanza della pre-elaborazione dei dati e dell'ottimizzazione degli iperparametri per la costruzione di modelli di *Machine Learning* robusti e generalizzabili avendo ottime prestazioni.

8 Support Vector Machine

L'algoritmo *Support Vector Machine* (SVM) è stato testato in diverse condizioni per analizzarne le prestazioni e ottimizzarne i parametri. Il processo è stato suddiviso in tre fasi principali: addestramento senza standardizzazione, con standardizzazione e ottimizzazione degli iperparametri tramite Grid Search.

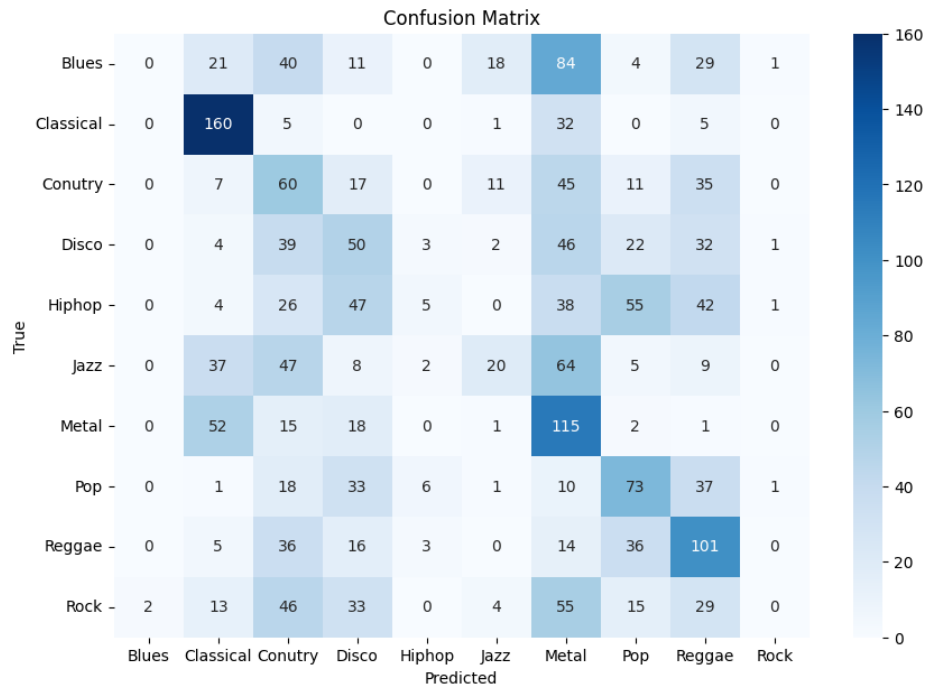
8.1 Fase 1: Addestramento Senza Standardizzazione

Nella prima fase, il modello SVM è stato addestrato senza alcuna standardizzazione delle feature, utilizzando un kernel *RBF* con $C = 0.9$ e $\gamma = \text{scale}$. L'accuratezza ottenuta su test è risultata inferiore rispetto a quella su training, suggerendo possibili problemi di *feature scaling*.

8.1.1 Vantaggi e Limiti

- **Vantaggi:** Modello semplice e rapido da addestrare.
- **Limiti:** SVM è sensibile alla scala delle feature; senza standardizzazione, le variabili con range numerici più ampi influenzano eccessivamente il margine di separazione, compromettendo le prestazioni del classificatore.

```
-----STATISTICHE TEST-----
Accuratezza: 0.2923
Precision: 0.2401
Recall: 0.2937
F1-score: 0.2365
F2-score: 0.2645
-----STATISTICHE TRAINING-----
Accuratezza: 0.2874
Precision: 0.2599
Recall: 0.2870
F1-score: 0.2337
F2-score: 0.2597
```



8.2 Fase 2: Addestramento con Standardizzazione

Per migliorare le prestazioni, è stata applicata la standardizzazione ai dati prima di addestrare nuovamente il modello. Questo passaggio normalizza le feature, garantendo che abbiano media zero e varianza unitaria.

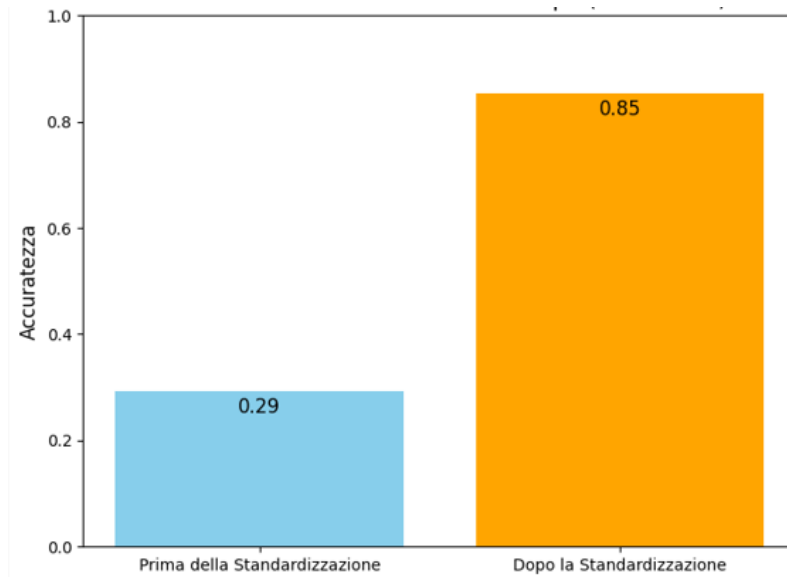
8.2.1 Risultati e Benefici

- **Aumento dell'accuratezza:** Dopo la standardizzazione, l'accuratezza del modello è migliorata rispetto alla fase precedente, come evidenziato dal confronto nei grafici.
- **Migliore convergenza:** La SVM sfrutta ottimamente la trasformazione dei dati, definendo margini di separazione più bilanciati.
- **Riduzione della sensibilità alla scala:** Il modello diventa più stabile e meno dipendente dalla distribuzione originale dei dati.

```

-----STATISTICHE TEST-----
Accuratezza: 0.8544
Precision: 0.8532
Recall: 0.8537
F1-score: 0.8522
F2-score: 0.8528
-----STATISTICHE TRAIING-----
Accuratezza: 0.9158
Precision: 0.9166
Recall: 0.9158
F1-score: 0.9158
F2-score: 0.9157

```



8.3 Fase 3: Ottimizzazione con Grid Search

Infine, è stata eseguita un'ottimizzazione degli iperparametri mediante *Grid Search*, con una ricerca su:

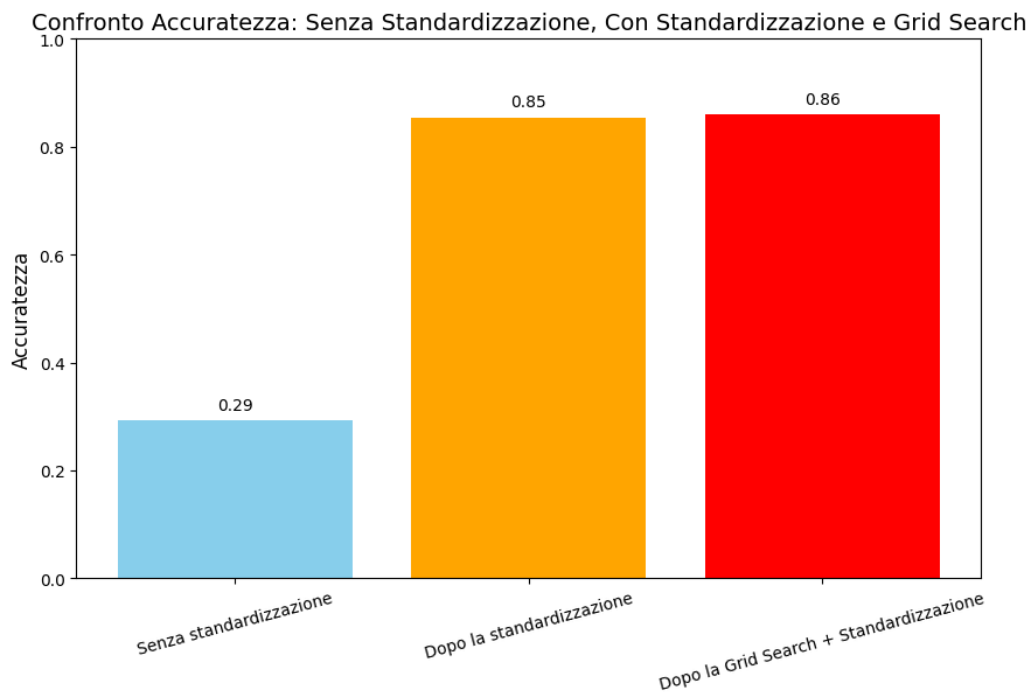
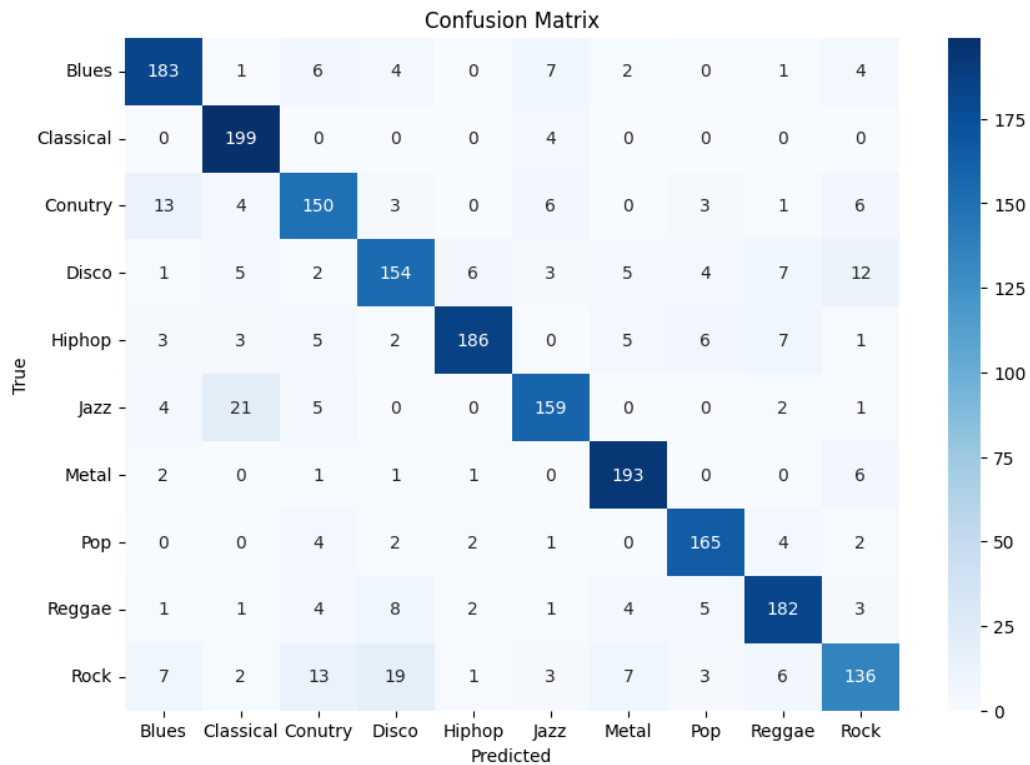
- Parametro di penalizzazione C (0.1, 1, 10).
- Scelta della funzione kernel (*rbf*, *poly*, *sigmoid*).
- Parametro γ (*scale*, *auto*).

Per selezionare il miglior modello, sono stati considerati solo i risultati con differenza di accuratezza tra training e test inferiore al 10%, evitando overfitting.

8.3.1 Vantaggi della Grid Search

- **Individuazione del miglior modello:** La combinazione ottimale dei parametri migliora sia accuratezza che generalizzazione.
- **Maggiore robustezza:** L'ottimizzazione riduce il rischio di scegliere iperparametri subottimali che potrebbero portare a overfitting o underfitting.
- **Bilanciamento tra bias e varianza:** La selezione dei parametri migliori assicura un compromesso ottimale tra la capacità predittiva sul training set e la generalizzazione sul test set.

```
Fitting 5 folds for each of 18 candidates, totalling 90 fits
Training Accuracy: 0.9220470470470471
Testing Accuracy: 0.8598598598598599
Best Parameters: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
La differenza tra l'accuracy di training e di test è entro il 10%.
```



8.4 Conclusioni

I risultati ottenuti mostrano che la standardizzazione è essenziale per migliorare le prestazioni di SVM, riducendo la sensibilità delle feature con range numerici differenti. Inoltre, l'ottimizzazione tramite Grid Search consente di ottenere un modello più efficiente, capace di generalizzare meglio sui dati di test. La metodologia adottata ha portato a un significativo miglioramento dell'accuratezza, mantenendo un divario contenuto tra training e test, garantendo un modello affidabile e ben bilanciato.

9 Extreme Gradient Boosting

L'algoritmo *Extreme Gradient Boosting* (XGBoost) è stato utilizzato per classificare il dataset e ottimizzato mediante Grid Search. L'analisi è stata suddivisa in tre fasi principali: addestramento senza standardizzazione, con standardizzazione e ottimizzazione degli iperparametri.

9.1 Fase 1: Addestramento Senza Standardizzazione

Il modello XGBoost è stato inizialmente addestrato senza alcuna standardizzazione delle feature. I parametri utilizzati includono:

- **learning rate** = 0.01
- **n estimators** = 600
- **max depth** = 9
- **min child weight** = 25
- **subsample** = 0.8
- **colsample bytree** = 0.8

Dopo l'addestramento, il modello è stato valutato su un test set, e i risultati hanno mostrato una discreta accuratezza.

9.1.1 Vantaggi e Limiti

- **Vantaggi:** La potenza del boosting permette di ottenere buoni risultati anche senza pre-processing avanzati.
- **Limiti:** Alta complessità computazionale, specialmente con dataset molto grandi

```
-----STATISTICHE TEST-----
Accuratezza: 0.8478
Precision: 0.8465
Recall: 0.8484
F1-score: 0.8464
F2-score: 0.8474
-----STATISTICHE TRAINING-----
Accuratezza: 0.9441
Precision: 0.9444
Recall: 0.9440
F1-score: 0.9440
F2-score: 0.9440
```

9.2 Fase 2: Addestramento con Standardizzazione

Sapendo che questa tecnica non è influenzata dalle scale dei parametri non ci si aspettavano grandi miglioramenti. Dopo questa trasformazione, il modello è stato riaddestrato con gli stessi parametri della fase precedente.

9.2.1 Risultati

- **Non si ha grande aumento dell'accuratezza:** Dopo la standardizzazione, l'accuratezza non è migliorata molto rispetto alla fase precedente, come ci si aspettava.

```
-----STATISTICHE TEST-----  
Accuratezza: 0.8509  
Precision: 0.8497  
Recall: 0.8513  
F1-score: 0.8495  
F2-score: 0.8503  
-----STATISTICHE TRAINING-----  
Accuratezza: 0.9439  
Precision: 0.9443  
Recall: 0.9439  
F1-score: 0.9439  
F2-score: 0.9439
```

9.3 Fase 3: Ottimizzazione con Grid Search

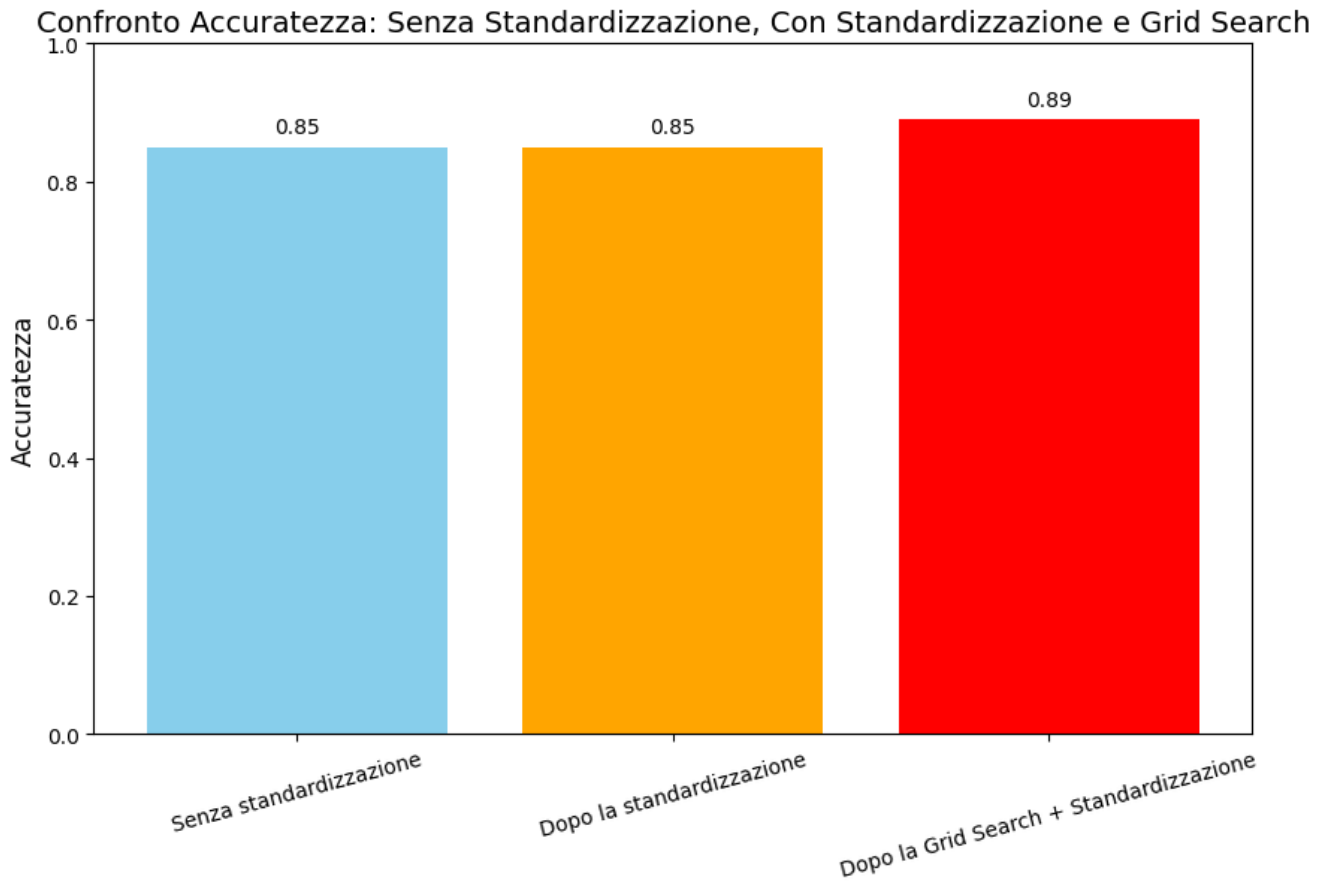
Successivamente, è stata eseguita un'ottimizzazione degli iperparametri tramite *Grid Search*, valutando diverse configurazioni:

- **Numero di alberi** (*n_estimators*): 100, 200.
- **Profondità massima degli alberi** (*max_depth*): 3, 5, 7.
- **Tasso di apprendimento** (*learning_rate*): 0.1, 0.3.

La validazione incrociata (*cross-validation*) con $k = 5$ è stata utilizzata per ridurre il rischio di overfitting e migliorare la capacità di generalizzazione.

9.3.1 Vantaggi della Grid Search

- **Individuazione del miglior modello:** La selezione dei migliori iperparametri ha permesso di ottenere la configurazione ottimale del modello.
- **Maggiore robustezza:** La ricerca su più combinazioni ha reso il modello meno soggetto a scelte arbitrarie di parametri subottimali.
- **Bilanciamento tra bias e varianza:** È stato adottato un criterio che filtra i modelli con differenze di accuratezza tra training e test superiori al 10%, evitando overfitting.



9.4 Conclusioni

I risultati ottenuti dimostrano che la standardizzazione non migliora le prestazioni del modello XGBoost. L'ottimizzazione degli iperparametri ha portato a un modello più efficiente e ben bilanciato, capace di garantire elevate prestazioni senza incorrere in overfitting. L'analisi comparativa tra le tre fasi conferma le aspettative, confermando l'importanza della selezione accurata degli iperparametri per massimizzare le performance del modello.

10 Semplice Rete Neurale Implementata

In questa sezione analizziamo l'implementazione e i risultati ottenuti utilizzando diverse architetture di reti neurali per la classificazione dei dati estratti dal dataset `features_3_sec.csv`. L'obiettivo è confrontare l'impatto di diversi design della rete sull'accuratezza e sulla capacità di generalizzazione del modello.

10.1 Pre-elaborazione dei Dati

Prima di addestrare i modelli, abbiamo effettuato una serie di operazioni di pre-processing essenziali per garantire una buona qualità dei dati in input:

- **Caricamento del dataset:** Il dataset è stato caricato e suddiviso in feature ($\mathbf{X}$) e target ($\mathbf{y}$), dove le feature rappresentano gli attributi numerici dei dati, mentre il target contiene le etichette di classificazione.

- **Codifica delle etichette:** Le etichette sono state trasformate in valori numerici utilizzando il Label Encoding, permettendo la compatibilità con i modelli di machine learning.
- **Standardizzazione:** Abbiamo applicato la *Z-score normalization* utilizzando lo StandardScaler di `sklearn`, che consente di scalare i dati con media zero e deviazione standard unitaria, migliorando la stabilità del training.
- **Suddivisione del dataset:** Il dataset è stato suddiviso in **70% training**, **20% validation** e **10% test**, garantendo un bilanciamento tra addestramento e valutazione.

10.2 Implementazione dei Modelli di Rete Neurale

Abbiamo sviluppato tre modelli di rete neurale con architetture progressive, ciascuno con diverse strategie per migliorare le prestazioni e ridurre l'overfitting.

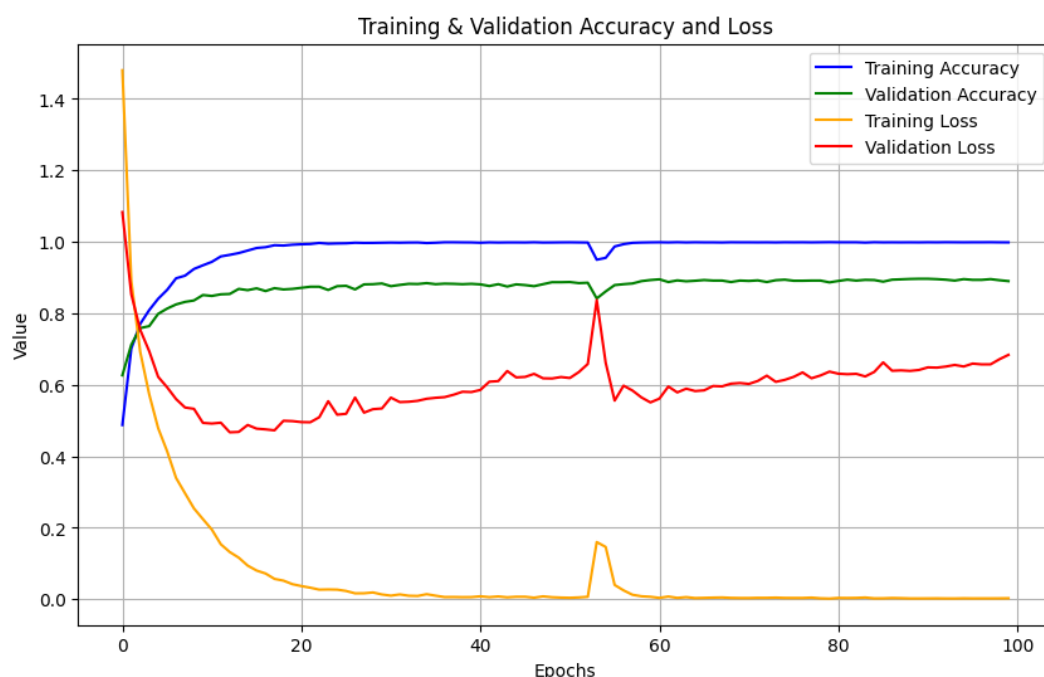
10.2.1 Modello 1 - Architettura di Base

Il primo modello è composto da:

- Tre strati completamente connessi (**Dense Layers**) con 256, 128 e 64 neuroni rispettivamente, tutti con funzione di attivazione ReLU.
- Uno strato di output con **softmax**, necessario per la classificazione multi-classe.

Vantaggi:

- Struttura semplice e veloce da addestrare.
- Buon punto di partenza per valutare la complessità necessaria del modello.



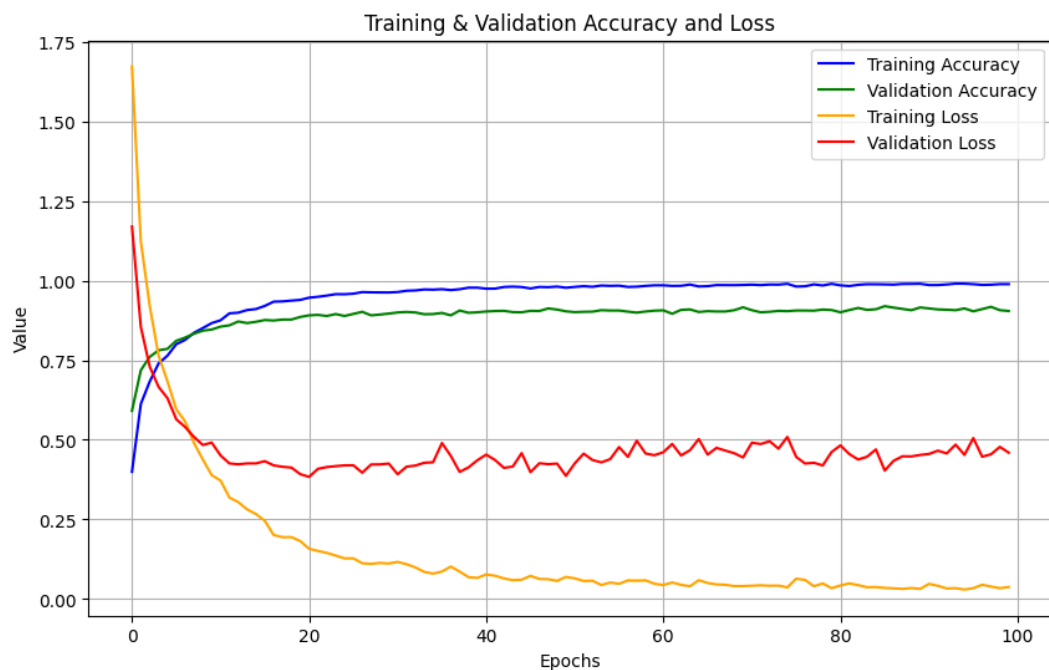
10.2.2 Modello 2 - Aggiunta di Dropout

Il secondo modello introduce la **Dropout Regularization**, con il seguente schema:

- Quattro livelli nascosti con 512, 256, 128 e 64 neuroni, ciascuno con attivazione ReLU.
- Dopo ogni strato nascosto, viene inserito un livello di dropout (20%) per ridurre il rischio di overfitting.

Vantaggi:

- Migliore capacità di generalizzazione rispetto al primo modello.
- Evita che la rete impari schemi troppo specifici ai dati di training.



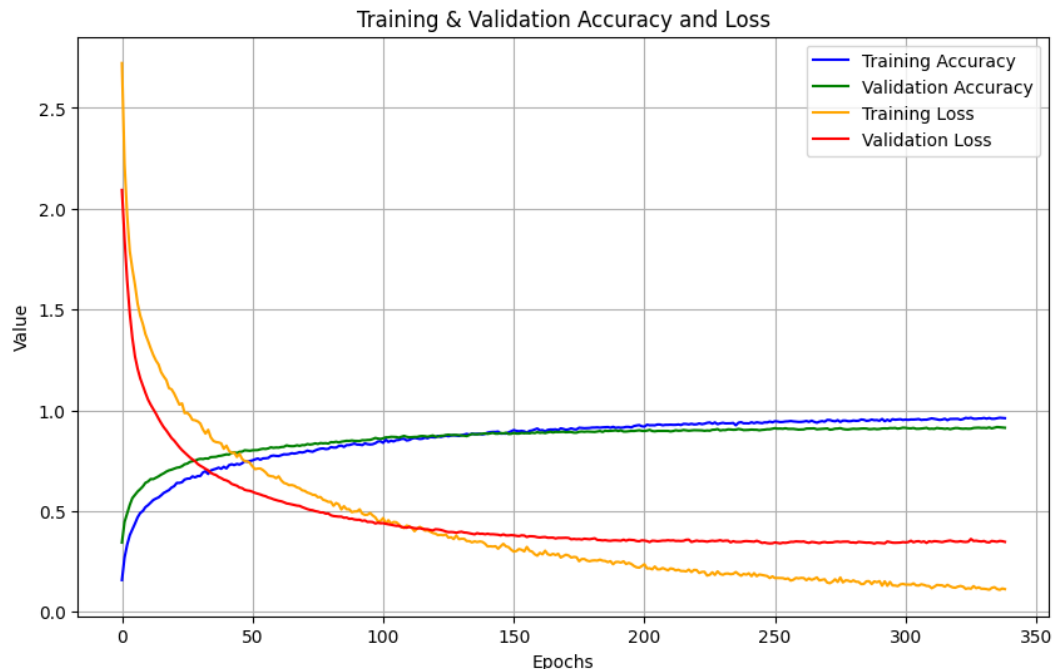
10.2.3 Modello 3 - Batch Normalization e Ottimizzazione Avanzata

Il terzo modello integra ulteriori tecniche avanzate per migliorare la stabilità e le performance:

- Cinque strati nascosti con **Batch Normalization** dopo ogni livello, per stabilizzare la distribuzione degli input e accelerare la convergenza.
- Utilizzo di **Dropout** con tassi più elevati (fino al 40%) per prevenire l'overfitting.
- Riduzione del **learning rate** a 0.0001 per un'ottimizzazione più controllata.
- **Early Stopping** per interrompere l'addestramento quando il modello non migliora più.

Vantaggi:

- Riduzione significativa del rischio di overfitting.
- Maggiore stabilità nel training grazie a Batch Normalization.
- Utilizzo ottimizzato delle risorse computazionali grazie all'early stopping.



10.3 Valutazione e Confronto dei Risultati

Tutti i modelli sono stati addestrati per un massimo di 100-400 epoche, utilizzando Adam come ottimizzatore e la funzione di perdita `sparse_categorical_crossentropy`. Le metriche di valutazione principali sono l'accuratezza di training, validation e test.

Il grafico finale mostra il confronto tra i tre modelli:

- **Modello 1:** Accuratezza moderata, ma incline all'overfitting nei dati di training.
- **Modello 2:** Buon equilibrio tra training e validation, ma ancora un margine di miglioramento nell'accuratezza di test.
- **Modello 3:** Ottimale bilanciamento tra accuratezza e capacità di generalizzazione.

I risultati confermano che l'introduzione di dropout, batch normalization e riduzione del learning rate porta a un miglioramento delle prestazioni del modello, con un'elevata accuratezza sul set di test.

10.4 Conclusioni

L'analisi ha dimostrato che l'uso di tecniche di regolarizzazione come Dropout e Batch Normalization, combinato con un tuning accurato del learning rate e l'early stopping, migliora significativamente le prestazioni delle reti neurali profonde. Il terzo modello risulta essere il più efficace, ottenendo un'elevata accuratezza senza overfitting, dimostrando la validità delle scelte architetturali adottate.

